

Containerization-Devops-Class

Containerization and DevOps

Name	Satvik Raj
SAP id	500119624
Batch	B1 CCVT

Lab Experiments

S. No.	Experiment
1	Comparison of Virtual Machines (VMs) and Containers
2	Docker Installation, Configuration, and Running Images
3	Deploying NGINX using different base images and comparing image layers
4	Docker Essentials
5	Docker - Volumes, Environment Variables, Monitoring & Networks
6	Comparison of Docker Run and Docker Compose
7	CI/CD using Jenkins, GitHub and Docker Hub
8	Ansible
9	Sonarqube
10	Orchestration using Docker Compose & Docker Swarm
11	Study and Analyse Container Orchestration using Kubernetes

Theory

- [Preserve changes in container](#)
- [Dockerfile Tutorial](#)
- [Python Script in docker](#)

- [Multistage docker build](#)
- [Data Management in Docker](#)
- [Networking in Docker](#)
- [Docker Composr](#)

Containerization-Devops-Class

Experiment - 1: Comparison of Virtual Machines (VMs) and Containers using Ubuntu and Nginx

Objective

1. To understand the conceptual and practical differences between Virtual Machines and Containers.
2. To install and configure a Virtual Machine using VirtualBox and Vagrant on Windows.
3. To install and configure Containers using Docker inside WSL.
4. To deploy an Ubuntu-based Nginx web server in both environments.
5. To compare resource utilization, performance, and operational characteristics of VMs and Containers.

Part A: Virtual Machine

- Download Vagrantfile of required VM OS using `vagrant init hashicorp/bionic64`, run `vagrant up` to start up the VM and `vagrant ssh` to access the VM terminal.

```
PS C:\Sunny\Study Materials\Contarization Lab> vagrant init hashicorp/bionic64
A 'Vagrantfile' has been placed in this directory. You are now
ready to 'vagrant up' your first virtual environment! Please read
the comments in the Vagrantfile as well as documentation on
'vagrantup.com' for more information on using Vagrant.
```

```
PS C:\Sunny\Study Materials\Contarization Lab> vagrant up
Bringing machine 'default' up with 'virtualbox' provider...
==> default: Box 'hashicorp/bionic64' could not be found. Attempting to find and install...
    default: Box Provider: virtualbox
    default: Box Version: >= 0
==> default: Loading metadata for box 'hashicorp/bionic64'
    default: URL: https://vagrantcloud.com/api/v2/vagrant/hashicorp/bionic64
==> default: Adding box 'hashicorp/bionic64' (v1.0.282) for provider: virtualbox
    default: Downloading: https://vagrantcloud.com/hashicorp/boxes/bionic64/versions/1.0.282/providers/virtualbox/unknown/vagrant.box
    default:
==> default: Successfully added box 'hashicorp/bionic64' (v1.0.282) for 'virtualbox'!
==> default: Importing base box 'hashicorp/bionic64'...
==> default: Matching MAC address for NAT networking...
==> default: Checking if box 'hashicorp/bionic64' version '1.0.282' is up to date...
==> default: Setting the name of the VM: ContarizationLab_default_1769245950432_29417
```

```

PS C:\Sunny\Study Materials\Contarization Lab> vagrant ssh
Welcome to Ubuntu 18.04.3 LTS (GNU/Linux 4.15.0-58-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

System information as of Sat Jan 24 09:15:04 UTC 2026

System load:  0.22           Processes:            91
Usage of /:   2.5% of 61.80GB Users logged in:          0
Memory usage: 11%           IP address for eth0: 10.0.2.15
Swap usage:   0%

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment

https://ubuntu.com/engage/secure-kubernetes-at-the-edge

0 packages can be updated.
0 updates are security updates.

vagrant@vagrant:~$ |

```

- Install nginx using `sudo apt install -y nginx` and run the service using `sudo systemctl start nginx`. Verify it by `curl localhost`.

```

vagrant@vagrant:~$ sudo apt update
Get:1 http://security.ubuntu.com/ubuntu bionic-security InRelease [102 kB]
Hit:2 http://archive.ubuntu.com/ubuntu bionic InRelease
Get:3 http://archive.ubuntu.com/ubuntu bionic-updates InRelease [102 kB]

vagrant@vagrant:~$ sudo apt install -y nginx
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following additional packages will be installed:
  fontconfig-config fonts-dejavu-core libfontconfig1 libgd3 libjpeg-turbo8 libjpeg8 libnginx-mod-http-geoip
  libnginx-mod-http-image-filter libnginx-mod-http-xslt-filter libnginx-mod-mail libnginx-mod-stream libtiff5 libwebp6
  libx11-6 libx11-data libxcb1 libxpm4 nginx-common nginx-core
Suggested packages:
  libgd-tools fcgiwrap nginx-doc ssl-cert
The following NEW packages will be installed:
  fontconfig-config fonts-dejavu-core libfontconfig1 libgd3 libjpeg-turbo8 libjpeg8 libnginx-mod-http-geoip
  libnginx-mod-http-image-filter libnginx-mod-http-xslt-filter libnginx-mod-mail libnginx-mod-stream libtiff5 libwebp6
  libx11-6 libx11-data libxcb1 libxpm4 nginx nginx-common nginx-core
0 upgraded, 21 newly installed, 0 to remove and 294 not upgraded.
Need to get 3,195 kB of archives.

```

```

vagrant@vagrant:~$ sudo systemctl start nginx
vagrant@vagrant:~$ curl localhost
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
    body {
        width: 35em;
        margin: 0 auto;
        font-family: Tahoma, Verdana, Arial, sans-serif;
    }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>

```

Observations

1. Storage Utilization: The disk usage for the VM.

```
VBoxManage showmediuminfo <path/to/virtualdrive>
```

```

In use by VMs:  ConVBoxManage showmediuminfo ".\VirtualBox VMs\ContainerizationLab_default_1769333222175_63647\ubuntu-18.04-amd64-disk001.vmdk"
UUID:          4993d570-6a33-4a41-b4cd-ab6e394b640f
Parent UUID:    base
State:          locked write
Type:           normal (base)
Location:       C:\Users\rajsa\VirtualBox VMs\ContainerizationLab_default_1769333222175_63647\ubuntu-18.04-amd64-disk001.vmdk
Storage format: VMDK
Format variant: dynamic default
Capacity:       65536 MBytes
Size on disk:   1773 MBytes
Encryption:     disabled
Property:       BootSector=
                Partitions=
                RawDrive=
                Relative=
In use by VMs:  ContainerizationLab_default_1769333222175_63647 (UUID: c1acee95-6fcb-46e1-8188-698e64cf38b4)

```

The VM installation consumed approximately **1773 MB** of disk space to store the Guest OS and virtual disk.

2. Boot Performance: The startup time required for the Virtual Machine to boot.

```
systemd-analyze
```

```
vagrant@vagrant:~$ systemd-analyze
Startup finished in 6.634s (kernel) + 20.414s (userspace) = 27.049s
graphical.target reached after 7.415s in userspace
```

The VM took **27.049 seconds** to fully boot (Kernel: 6.634s + Userspace: 20.414s), demonstrating high startup latency.

3. Memory Usage: Amount of RAM resources allocated to and used by the Guest OS.

```
free -h
```

	total	used	free	shared	buff/cache	available
Mem:	985M	73M	242M	5.0M	670M	763M
Swap:	979M	0B	979M			

The VM reserved **985 MB** of total RAM from the host, with **73 MB** actively used and 670 MB used up in buffers/cache.

Part B: Containers

- Run the `docker run -d -p 8080:80 --name nginx-container nginx` command to create and run the container. Verify that the nginx container is running with `curl localhost:8080`.
 - `-d` : Runs the container in the background, allowing you to continue using the terminal session.
 - `-p 8080:80` : Maps port 8080 for the host to port 80 for the container.

```
> docker run -d -p 8080:80 --name nginx-container nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
eaf8753feae0: Pull complete
d989100b8a84: Pull complete
700146c8ad64: Pull complete
500799c30424: Pull complete
119d43eec815: Pull complete
10b68cfefee1: Pull complete
57f0dd1befe2: Pull complete
785250c9bf9e: Download complete
e2dd2dbe6277: Download complete
Digest: sha256:c881927c4077710ac4b1da63b83aa163937fb47457950c267d92f7e4dedf4aec
Status: Downloaded newer image for nginx:latest
d348c9d509e3f96cec0c201e0258c7d2cdbd0ec5f0bf84e86081b91434e251b1
```

```
> curl localhost:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

Observations

1. Storage Utilization An analysis of the disk space required for the container images.

docker images

```
sunny@Sunny-A15:~$ docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
ubuntu        latest    493218ed0f40   11 days ago   78.1MB
nginx         latest    4af177a024eb   11 days ago   161MB
```

The Nginx container image requires only **161 MB**.

2. Boot Performance Measurement of the time required to start the containerized application.

```
time docker run -d -p 8080:80 --name nginx-container nginx
```

```
sunny@Sunny-A15:~$ time docker run -d -p 8080:80 --name nginx-container nginx
4bc575f34574519af1713d3a10b325d4e2057132b3c6988a3dab0cc05f4d5e21

real    0m0.428s
user    0m0.012s
sys     0m0.042s
```

The container started in **0.428 seconds**.

3. Memory Allocation Real-time monitoring of the container’s resource consumption.

```
docker stats
```

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
4bc575f34574	nginx-container	0.00%	13.14MiB / 7.381GiB	0.17%	806B / 126B	0B / 12.3kB	17

The running container uses **13.14 MiB** of RAM.

Comparison

Metric	Virtual Machine	Container	Difference
Disk Usage	~1.8 GB	161 MB	Containers are <i>smaller</i> because they do not require a full Guest OS copy.
Boot Time	27.05s	0.43s	Containers start <i>faster</i> by leveraging the already-running Host Kernel.

Metric	Virtual Machine	Container	Difference
Memory	~985 MB Reserved / 73 MB Used	13.14 MB Used	VMs reserve a fixed block of RAM; Containers use only what the process needs dynamically.
Architecture	Hardware Virtualization (Guest OS)	OS Virtualization (Shared Kernel)	VMs run a full isolated OS; Containers run as isolated processes on the Host OS.

Containerization-Devops-Class

Experiment 2: Docker Installation, Configuration and Image Management

Objective:

1. Pull docker images
2. Run Containers
3. Manage container lifecycle

Procedure

1. Pull Image

```
docker pull nginx
```

```
> docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
eaf8753feae0: Pull complete
500799c30424: Pull complete
10b68cfefee1: Pull complete
57f0dd1befe2: Pull complete
119d43eec815: Pull complete
700146c8ad64: Pull complete
d989100b8a84: Pull complete
e2dd2dbe6277: Download complete
785250c9bf9e: Download complete
Digest: sha256:c881927c4077710ac4b1da63b83aa163937fb47457950c267d92f7e4dedf4aec
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
```

2. Running the container with port mapping.

```
docker run -d -p 8080:80 nginx
```

```
> docker run -d -p 8080:80 nginx
3a113f60ce5b21b65c0745bfff874e42b3b683e363e3b2037a14853dfecb93a2
```

- -d (detached): Runs the container in the background

- -p 8080:80 (publish): Connects a port on host to a port inside the container.
- 8080 : Host's Port
- 80 : Container's port - The output is the ID of the container.

3. Verifying the running container.

```
docker ps
```

```
> docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
3a113f60ce5b   nginx    "/docker-entrypoint..." 48 seconds ago Up 47 seconds 0.0.0.0:8080→80/tcp, [::]:8080→80/tcp sad_perlman
```

4. Stop the running container, then remove it.

```
docker stop <container_id>
```

```
> docker stop 3a11
3a11
> docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
3a113f60ce5b   nginx    "/docker-entrypoint..." 9 minutes ago  Exited (0) 40 seconds ago                               sad_perlman
```

```
docker rm <container_id>
```

```
> docker rm 3a11
3a11
> docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
```

5. Remove image

```
docker rmi <image_name>
```

```
IMAGE          ID              DISK USAGE  CONTENT SIZE  EXTRA
nginx:latest    c881927c4077    239MB       65.7MB
> docker rmi nginx
Untagged: nginx:latest
Deleted: sha256:c881927c4077710ac4b1da63b83aa163937fb47457950c267d92f7e4dedf4aec
```

Result

Docker images were successfully pulled, containers executed, and lifecycle commands performed.

Containerization-Devops-Class

Experiment 3: Deploying NGINX using different base images and comparing image layers

Objective:

1. Deploy NGINX using: i. Official `nginx` image ii. Ubuntu-based image iii. Alpine-based image
2. Understand Docker image layers and size differences
3. Compare performance, security, and use-cases

Part 1: Deploy NGINX Using Official Image

To pull and build the official NGINX image:

```
docker pull nginx:latest
docker run -d --name nginx-official -p 8080:80 nginx
```

Verify the installation using:

```
curl http://localhost:8080
```

The official NGINX image:

```
> docker images nginx
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
nginx:latest	341bf0f3ce6c	239MB	65.8MB	

Part 2: Custom NGINX using Ubuntu Base Image

Create a docker file *ubuntu.Dockerfile*:

```
FROM ubuntu:latest
RUN apt-get update && \
    apt-get install -y nginx && \
    apt-get clean && \
    rm -rf /var/lib/apt/lists/*
```

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]

Build and run the nginx-ubuntu image (running on port 8081):

```
docker build -t nginx-ubuntu -f ./ubuntu.Dockerfile .
docker run -d -v ./nginx-html:/var/www/html -p 8081:80 --name nginx-ubuntu-test nginx
```

NOTE: Assuming *index.html* is stored in *./nginx-html*.

```
> docker build -t nginx-ubuntu -f ./ubuntu.Dockerfile .
[+] Building 55.3s (6/6) FINISHED                                docker:default
=> [internal] load build definition from ubuntu.Dockerfile       0.0s
=> => transferring dockerfile: 223B                               0.0s
=> [internal] load metadata for docker.io/library/ubuntu:latest 0.1s
=> [internal] load .dockerignore                                  0.0s
=> => transferring context: 2B                                     0.0s
=> [1/2] FROM docker.io/library/ubuntu:latest@sha256:cd1dba651b3080c3686ecf4e3c4220f026b521fb76978881737d24f200828 0.1s
=> => resolve docker.io/library/ubuntu:latest@sha256:cd1dba651b3080c3686ecf4e3c4220f026b521fb76978881737d24f200828 0.1s
=> [2/2] RUN apt-get update && apt-get install -y nginx && apt-get clean && rm -rf /var/lib/apt/lists 54.1s
=> exporting to image                                           0.7s
=> => exporting layers                                           0.4s
=> => exporting manifest sha256:fcd4b47c6b7e6a635ec67bcd8c526508cda11842e1ad0d4681990a5dd13e7e 0.0s
=> => exporting config sha256:3749d9389c50cb30fa6ca491f9eb4d43d26012b8d3c721b00ffee236de3dc893 0.0s
=> => exporting attestation manifest sha256:bd0aacb3fdcda8e835df4a40eb9b51e313e0f576c75f374f34ea23a97e838ce5 0.1s
=> => exporting manifest list sha256:4d404970a39030d5e0e73231b6a5be2b8596d93cd70eb90ea24ab8b05cc757f4 0.0s
=> => naming to docker.io/library/nginx-ubuntu:latest           0.0s
=> => unpacking to docker.io/library/nginx-ubuntu:latest        0.1s
```

The built Ubuntu NGINX image:

```
> docker images nginx-ubuntu
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA	In Use
nginx-ubuntu:latest	4d404970a390	125MB	32.2MB	U	

Part 3: Custom NGINX Using Alpine Base Image

Create a docker file *alpine.Dockerfile*:

```
FROM alpine:latest
RUN apk add --no-cache nginx
RUN echo "server { \
    listen 80 default_server; \
    listen [::]:80 default_server; \
    root /var/www/html; \
    index index.html index.htm; \
    server_name _; \
    location / { \
        try_files \$uri \$uri/ =404; \
    } \
}
```

```
} " > /etc/nginx/http.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

NOTE: The default config of NGINX on Alpine is set to display a 404 page.

Build and run the nginx-alpine image (running on port 8082):

```
docker build -t nginx-alpine -f ./alpine.Dockerfile .
docker run -d -v ./nginx-html:/var/www/html -p 8082:80 --name nginx-alpine-test nginx
```

```
> docker build -t nginx-alpine -f ./alpine.Dockerfile .
[+] Building 0.5s (7/7) FINISHED                                docker:default
=> [internal] load build definition from alpine.Dockerfile      0.0s
=> => transferring dockerfile: 411B                             0.0s
=> [internal] load metadata for docker.io/library/alpine:latest 0.1s
=> [internal] load .dockerignore                               0.0s
=> => transferring context: 2B                                    0.0s
=> [1/3] FROM docker.io/library/alpine:latest@sha256:25109184c71bdad752c8312a8623239686 0.1s
=> => resolve docker.io/library/alpine:latest@sha256:25109184c71bdad752c8312a8623239686 0.0s
=> CACHED [2/3] RUN apk add --no-cache nginx                    0.0s
=> CACHED [3/3] RUN echo "server {      listen 80 default_server;      listen [::]:80 def 0.0s
=> exporting to image                                          0.2s
=> => exporting layers                                          0.0s
=> => exporting manifest sha256:e12c8d9911124174feb389ee971391deb625271c1814d0f796d1e0c 0.0s
=> => exporting config sha256:bb6e174510f9b8099231c8bdfc8cf90ba2e1c18558eafc92f1f926310 0.0s
=> => exporting attestation manifest sha256:7f1135240b466afec47d52dad37d74783e26220a147 0.1s
=> => exporting manifest list sha256:dd2af5e3676c63c517d7a4d3dced63332a150ca4f024d346dc 0.0s
=> => naming to docker.io/library/nginx-alpine:latest          0.0s
=> => unpacking to docker.io/library/nginx-alpine:latest       0.0s
```

The built Alpine NGINX image:

```
> docker images nginx-alpine
```

IMAGE	ID	DISK USAGE	CONTENT SIZE
nginx-alpine:latest	dd2af5e3676c	16.9MB	4.74MB

Part 4: Image Size and Layer Comparison

Compare Sizes

```
docker images | grep nginx
```

nginx-alpine:latest	dd2af5e3676c	16.9MB	4.74MB
nginx-ubuntu:latest	7db823f01427	125MB	32.2MB
nginx:latest	341bf0f3ce6c	239MB	65.8MB

Image	Size
nginx	239MB
nginx-ubuntu	125MB
nginx-alpine	16.9MB

Comapare Layers

```
docker history nginx
```

```
> docker history nginx
```

IMAGE	CREATED	CREATED BY	SIZE	COMMENT
341bf0f3ce6c	3 days ago	CMD ["nginx" "-g" "daemon off;"]	0B	buildkit.dockerfile.v0
<missing>	3 days ago	STOPSIGNAL SIGQUIT	0B	buildkit.dockerfile.v0
<missing>	3 days ago	EXPOSE map[80/tcp:{}]	0B	buildkit.dockerfile.v0
<missing>	3 days ago	ENTRYPOINT ["/docker-entrypoint.sh"]	0B	buildkit.dockerfile.v0
<missing>	3 days ago	COPY 30-tune-worker-processes.sh /docker-ent...	8.19kB	buildkit.dockerfile.v0
<missing>	3 days ago	COPY 20-envsubst-on-templates.sh /docker-ent...	4.1kB	buildkit.dockerfile.v0
<missing>	3 days ago	COPY 15-local-resolvers.envsh /docker-entryp...	4.1kB	buildkit.dockerfile.v0
<missing>	3 days ago	COPY 10-listen-on-ipv6-by-default.sh /docker...	4.1kB	buildkit.dockerfile.v0
<missing>	3 days ago	COPY docker-entrypoint.sh / # buildkit	4.1kB	buildkit.dockerfile.v0
<missing>	3 days ago	RUN /bin/sh -c set -x && groupadd --syst...	87.2MB	buildkit.dockerfile.v0
<missing>	3 days ago	ENV DYNPKG_RELEASE=1~trixie	0B	buildkit.dockerfile.v0
<missing>	3 days ago	ENV PKG_RELEASE=1~trixie	0B	buildkit.dockerfile.v0
<missing>	3 days ago	ENV ACME_VERSION=0.3.1	0B	buildkit.dockerfile.v0
<missing>	3 days ago	ENV NJS_RELEASE=1~trixie	0B	buildkit.dockerfile.v0
<missing>	3 days ago	ENV NJS_VERSION=0.9.5	0B	buildkit.dockerfile.v0
<missing>	3 days ago	ENV NGINX_VERSION=1.29.5	0B	buildkit.dockerfile.v0
<missing>	3 days ago	LABEL maintainer=NGINX Docker Maintainers <d...	0B	buildkit.dockerfile.v0
<missing>	6 days ago	# debian.sh --arch 'amd64' out/ 'trixie' '@1...	86.4MB	debuerreotype 0.17

```
docker history nginx-ubuntu
```

```
> docker history nginx-ubuntu
```

IMAGE	CREATED	CREATED BY	SIZE	COMMENT
7db823f01427	56 minutes ago	CMD ["nginx" "-g" "daemon off;"]	0B	buildkit.dockerfile.v0
<missing>	56 minutes ago	EXPOSE [80/tcp]	0B	buildkit.dockerfile.v0
<missing>	56 minutes ago	RUN /bin/sh -c apt-get update && apt-get...	7.12MB	buildkit.dockerfile.v0
<missing>	3 weeks ago	/bin/sh -c #(nop) CMD ["/bin/bash"]	0B	
<missing>	3 weeks ago	/bin/sh -c #(nop) ADD file:3077ee44db3cc7d38...	85.7MB	
<missing>	3 weeks ago	/bin/sh -c #(nop) LABEL org.opencontainers...	0B	
<missing>	3 weeks ago	/bin/sh -c #(nop) LABEL org.opencontainers...	0B	
<missing>	3 weeks ago	/bin/sh -c #(nop) ARG LAUNCHPAD_BUILD_ARCH	0B	
<missing>	3 weeks ago	/bin/sh -c #(nop) ARG RELEASE	0B	

```
docker history nginx-alpine
```

```

> docker history nginx-alpine
IMAGE          CREATED          CREATED BY          SIZE      COMMENT
dd2af5e3676c   About an hour ago  CMD ["nginx" "-g" "daemon off;"]  0B        buildkit.dockerfile.v0
<missing>      About an hour ago  EXPOSE [80/tcp]      0B        buildkit.dockerfile.v0
<missing>      About an hour ago  RUN /bin/sh -c echo "server {      listen 80 ...  4.1kB     buildkit.dockerfile.v0
<missing>      About an hour ago  RUN /bin/sh -c apk add --no-cache nginx # bu...  2.1MB     buildkit.dockerfile.v0
<missing>      11 days ago       CMD ["/bin/sh"]      0B        buildkit.dockerfile.v0
<missing>      11 days ago       ADD alpine-minirootfs-3.23.3-x86_64.tar.gz /...  10.1MB    buildkit.dockerfile.v0

```

Observations

- Size of official nginx image is the largest (239MB).
- nginx-alpine is the smallest (125MB).
- Official nginx image has the most layers.
- nginx-alpine has the least layers.

Part 5: Functional Tasks Using NGINX

Serving custom HTML page

./nginx-html/index.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Student Information</title>
  <style>
    body {
      display: flex;
      justify-content: center;
      padding-top: 50px;
    }
    th, td {
      border: 1px solid #dddddd;
      text-align: left;
      padding: 12px;
    }
  </style>
</head>
<body>

<table>
  <thead>
    <tr>
      <th>Field</th>
      <th>Details</th>
    </tr>
  </thead>

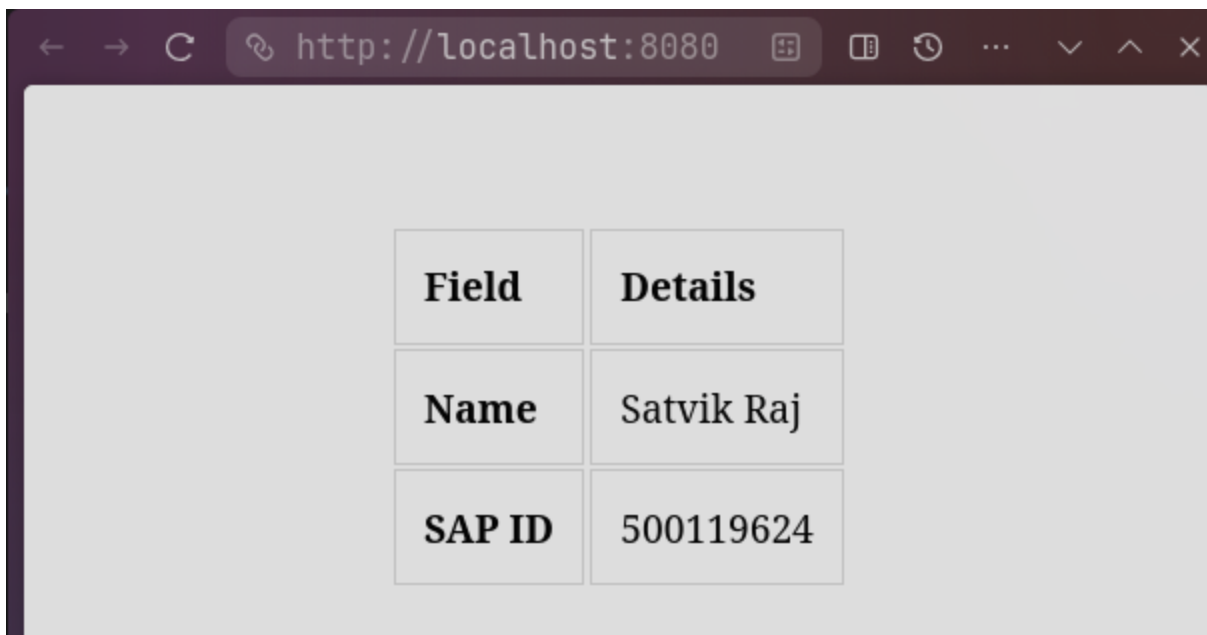
```

```
<tbody>
  <tr>
    <td><strong>Name</strong></td>
    <td>Satvik Raj</td>
  </tr>
  <tr>
    <td><strong>SAP ID</strong></td>
    <td>500119624</td>
  </tr>
</tbody>
</table>

</body>
</html>
```

Running on official NGINX image:

```
docker run -d \
  -p 8080:80 \
  -v ./nginx-html:/usr/share/nginx/html \
  nginx
```



Part 6: Set up a Flask Server

flask-alpine.Dockerfile:

```
FROM python:3.11.14-alpine3.23
WORKDIR /home
```

```
RUN pip install flask
CMD ["python", "./app.py"]
```

app.py:

```
from flask import Flask, render_template

app = Flask(__name__)

@app.route('/')
def hello():
    return "Hello from Docker!"

@app.route('/health')
def health():
    return "OK"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Containerization-Devops-Class

Experiment 4: Docker Essentials

Dockerfile **.dockerignore** **tagging** **publishing**

Part 1: Containerizing Applications with Dockerfile

Step 1: Create Python Flask App:

```
mkdir python-app  
cd python-app
```

app.py :

```
from flask import Flask  
app = Flask(__name__)  
  
@app.route('/')  
def hello():  
    return "Hello from Docker!"  
  
@app.route('/health')  
def health():  
    return "OK"  
  
if __name__ == '__main__':  
    app.run(host='0.0.0.0', port=5000)
```

requirements.txt :

```
blinker==1.9.0  
click==8.3.1  
Flask==3.1.3  
itsdangerous==2.2.0  
Jinja2==3.1.6
```

```
MarkupSafe==3.0.3
Werkzeug==3.1.6
```

Step 2: Create Dockerfile

Dockerfile :

```
FROM python:3.13-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

Step 3: Basic Build Command

```
# Tag with version number
docker build -t my-flask-app:1.0 .

# Tag with multiple tags
docker build -t my-flask-app:latest -t my-flask-app:1.0 .

# Tag with custom registry
docker build -t username/my-flask-app:1.0 .

# Tag existing image
docker tag my-flask-app:latest my-flask-app:v1.0
```

Step 4: View Image Details

```
# List all images
docker images

# Show image history
docker history my-flask-app

# Inspect image details
docker inspect my-flask-app
```

Part 2: Running Containers

Step 1: Run Container

```
# Run container with port mapping
docker run -d -p 5000:5000 --name flask-container my-flask-app

# Test the application
curl http://localhost:5000

# View running containers
docker ps

# View container logs
docker logs flask-container
```

Step 2: Manage Containers

```
# Stop container
docker stop flask-container

# Start stopped container
docker start flask-container

# Remove container
docker rm flask-container

# Remove container forcefully
docker rm -f flask-container
```

Part 3: Multi-stage Builds

Why Multi-stage Builds?

- Smaller final image size
- Better security (remove build tools)
- Separate build and runtime environments

Step 1: Simple Multi-stage Dockerfile

Dockerfile.multistage :

```
FROM python:3.13-slim AS builder
WORKDIR /app
```

```

COPY requirements.txt .
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
RUN pip install --no-cache-dir -r requirements.txt

FROM python:3.13-slim
WORKDIR /app
COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
COPY app.py .
RUN useradd -m -u 1000 appuser
USER appuser
EXPOSE 5000
CMD ["python", "app.py"]

```

Step 3: Build and Compare

```

# Build regular image
docker build -t flask-regular .

# Build multi-stage image
docker build -f Dockerfile.multistage -t flask-multistage .

# Compare sizes
docker images | grep flask-

```

flask-test:latest	6e96214c51b6	474MB	132MB
flask-test:multistage	92870a23221c	205MB	49.9MB
rajsunny004/flask-test:latest	5c4c708385ad	198MB	48.8MB

Saved **56.75%** in image size with multistage build.

Part 6: Publishing to Docker Hub

Step 1: Prepare for Publishing

```

# Login to Docker Hub
docker login

# Tag image for Docker Hub
docker tag flask-test:latest rajsatvik004/flask-test:latest

```

```
# Push to Docker Hub
docker push rajsatvik004/flask-test:latest
```

```
> docker push rajsunny004/flask-test:latest
The push refers to repository [docker.io/rajsunny004/flask-test]
76252be3e1d1: Pushed
b3ec39b36ae8: Pushed
3f8773a1d98b: Pushed
fc7443084902: Pushed
38513bd72563: Pushed
94c78ca8e9ac: Pushed
8b4a18032991: Pushed
b3ed828a7e3e: Pushed
ea56f685404a: Pushed
latest: digest: sha256:5c4c708385ada81b947bec191aca989927641d3dce338c2e460e20902f1ba4cf size: 856
```

Step 2: Pull and Run from Docker Hub

```
# Pull from Docker Hub (on another machine)
docker pull rajsatvik004/flask-test:latest

# Run the pulled image
docker run -d -p 5000:5000 rajsatvik004/flask-test:latest
```

Part 7: Node.js Example (Quick Version)

Step 1: Node.js Application

```
mkdir my-node-app
cd my-node-app
```

app.js :

```
const express = require('express');
const app = express();
const port = 3000;

app.get('/', (req, res) => {
  res.send('Hello from Node.js Docker!');
});

app.get('/health', (req, res) => {
  res.json({ status: 'healthy' });
});
```

```
app.listen(port, () => {  
  console.log(`Server running on port ${port}`);  
});
```

package.json :

```
{  
  "name": "node-docker-app",  
  "version": "1.0.0",  
  "main": "app.js",  
  "dependencies": {  
    "express": "^4.18.2"  
  }  
}
```

Step 2: Node.js Dockerfile

```
FROM node:18-alpine  
WORKDIR /app  
COPY package*.json ./  
RUN npm install --only=production  
COPY app.js .  
EXPOSE 3000  
CMD ["node", "app.js"]
```

Step 3: Build and Run

```
# Build image  
docker build -t my-node-app .  
  
# Run container  
docker run -d -p 3000:3000 --name node-container my-node-app  
  
# Test  
curl http://localhost:3000
```

```
docker build -t nodejs-test:1 -f Dockerfile.node-app .
[+] Building 24.9s (11/11) FINISHED
=> [internal] load build definition from Dockerfile.node-app
=> => transferring dockerfile: 185B
=> [internal] load metadata for docker.io/library/node:18-alpine
[auth] library/node:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d18945588d0a35cb9bc4b8ca09d9e
=> => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d18945588d0a35cb9bc4b8ca09d9e
=> sha256:25ff2da83641908f65c3a74d80409d6b1b62ccfaab220b9ea70b80df5a2e0549 446B / 446B
=> => sha256:1e5a4c89cee5c0826c540ab06d4b6b491c96eda01837f430bd47f0d26702d6e3 1.26MB / 1.26MB
=> sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4efea161b2b5a3d0e 40.01MB / 40.01MB
=> sha256:f18232174bc91741fd3da96d85011092101a032a93a388b79e99e69c2d5c870 3.64MB / 3.64MB
=> => extracting sha256:f18232174bc91741fd3da96d85011092101a032a93a388b79e99e69c2d5c870
=> => extracting sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4efea161b2b5a3d0e
=> => extracting sha256:1e5a4c89cee5c0826c540ab06d4b6b491c96eda01837f430bd47f0d26702d6e3
=> => extracting sha256:25ff2da83641908f65c3a74d80409d6b1b62ccfaab220b9ea70b80df5a2e0549
=> [internal] load build context
=> => transferring context: 514B
=> [2/5] WORKDIR /app
=> [3/5] COPY package*.json ./
=> [4/5] RUN npm install --only=production
=> [5/5] COPY app.js .
=> => exporting to image
=> => exporting layers
=> => exporting manifest sha256:ed33c0998e6a468112b647995e43827842f0226e1d3927c63534322aab3efef0
=> => exporting config sha256:c7b2a1fb5be23ffce0e16f2dea27f1cd7ddc44951300a5a7a6c6eaf60ccfd4fa1e
=> => exporting attestation manifest sha256:1096f4437402a23b0e8b9787969237fa26421a7cfa2f46d9378a9e871f8173b
=> => exporting manifest list sha256:ba78e7635a7c66be14c37581c54e164f69847d021295377353534a0d54082a93
=> => naming to docker.io/library/nodejs-test:1
=> => unpacking to docker.io/library/nodejs-test:1
```

```
docker run -d -p 3000:3000 --name node-container nodejs-test:1
c3e249207ad3227bfff5fad043d1231995de57df714aaac06a7693813d35a7d8d
curl http://localhost:3000
Hello from Node.js Docker!%
```

Essential Docker Commands Cheatsheet

Command	Purpose	Example
docker build	Build image	docker build -t myapp .
docker run	Run container	docker run -p 3000:3000 myapp
docker ps	List containers	docker ps -a
docker images	List images	docker images
docker tag	Tag image	docker tag myapp:latest myapp:v1
docker login	Login to Dockerhub using username and password or token	echo "token" \ docker login -u username --password-stdin
docker push	Push to registry	docker push username/myapp
docker pull	Pull from registry	docker pull username/myapp
docker rm	Remove container	docker rm container-name

Command	Purpose	Example
<code>docker rmi</code>	Remove image	<code>docker rmi image-name</code>
<code>docker logs</code>	View logs	<code>docker logs container-name</code>
<code>docker exec</code>	Execute command	<code>docker exec -it container-name bash</code>

Containerization-Devops-Class

Experiment 5: Docker - Volumes, Environment Variables, Monitoring & Networks

Part 1: Docker Volumes - Persistent Data Storage

Lab 1: Understanding Data Persistence

The Problem: Container Data is Ephemeral

```
docker run -it --name test-container ubuntu /bin/bash

mkdir -p /data && echo "Hello World" > /data/message.txt
cat /data/message.txt # Shows "Hello World"
exit

docker start test-container
docker exec test-container cat /data/message.txt # Shows "Hello World"
```

Data isn't lost if the container isn't removed.

```
docker run -it --rm --name test-container ubuntu /bin/bash

# In container
mkdir -p /data && echo "Hello World" > /data/message.txt
cat /data/message.txt # Shows "Hello World"
exit

docker run -it --rm --name test-container ubuntu /bin/bash

# In container
cat /data/message.txt # Error: No such file or directory
exit
```

```

) docker rm -f test-container
test-container
) docker run -it --rm --name test-container ubuntu /bin/bash
root@8b54b310ca27:/# mkdir -p /data && echo "Hello World" > /data/message.txt
root@8b54b310ca27:/# cat /data/message.txt
Hello World
root@8b54b310ca27:/# exit
exit
) docker run -it --rm --name test-container ubuntu /bin/bash
root@bcad2018e42f:/# cat /data/message.txt
cat: /data/message.txt: No such file or directory
root@bcad2018e42f:/# exit
exit

```

Lab 2: Volume Types

1. Anonymous Volumes

```
docker run -d -v /app/data --name web1 nginx
```

```
# Check volume
```

```
docker volume ls
```

```
# Shows: anonymous volume with random hash
```

```
# Inspect container to see volume mount
```

```
docker inspect web1 | grep -A 5 Mounts
```

```

) docker run -d -v /app/data --name web1 nginx
633d5a742f07313d791f29b2b07d2d13f057c836cfc962c7cfe90ab0120aff64
) docker volume ls
DRIVER      VOLUME NAME
local      58a5fe9ab864b395a4684d3cf757d29e69e8b3325d997d9099d0d82932db3239

```

2. Named Volumes

```
# Create named volume
```

```
docker volume create mydata
```

```
# Use named volume
```

```
docker run -d -v mydata:/app/data --name web2 nginx
```

```
# List volumes
```

```
docker volume ls
```

```
# Shows: mydata
```

```
# Inspect volume
docker volume inspect mydata
```

```
> docker volume rm mydata
mydata
> docker run -d -v mydata:/app/data --name web2 nginx
74ebf4db8b5ccd46339eddc17c230faa662c16c9391821fa2770eb24fc01a8ad
> docker volume ls
DRIVER      VOLUME NAME
local       58a5fe9ab864b395a4684d3cf757d29e69e8b3325d997d9099d0d82932db3239
local       mydata
> docker volume inspect mydata
[
  {
    "CreatedAt": "2026-03-20T09:31:13+05:30",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/mydata/_data",
    "Name": "mydata",
    "Options": null,
    "Scope": "local"
  }
]
```

3. Bind Mounts (Host Directory)

```
# Create directory on host
mkdir ./myapp-data

# Mount host directory to container
docker run -d -v ./myapp-data:/app/data --name web3 nginx

# Add file on host
echo "From Host" > ./myapp-data/host-file.txt

# Check in container
docker exec web3 cat /app/data/host-file.txt
# Shows: From Host
```

```
> mkdir myapp-data
> docker run -d -v ./myapp-data:/app/data --name web3 nginx
eae73b8af5471cd904a06fd1afc08dbfc75dda120074697bd6c4cd28de20620a
> echo "From Host" > ./myapp-data/host-file.txt
> docker exec web3 cat /app/data/host-file.txt
From Host
```

Lab 3: Volume Management Commands

List all volumes

```
docker volume ls
```

Create a volume

```
docker volume create app-volume
```

Inspect volume details

```
docker volume inspect app-volume
```

Remove unused volumes

```
docker volume prune
```

Remove specific volume

```
docker volume rm volume-name
```

Copy files to/from volume

```
docker cp local-file.txt container-name:/path/in/volume
```

Containerization-Devops-Class

Experiment 6: Comparison of Docker Run and Docker Compose

A. Theory

1. Objective

To understand the relationship between `docker run` and Docker Compose, and to compare their configuration syntax and use cases.

2. Background Theory

2.1 Docker Run (Imperative Approach)

The `docker run` command is used to create and start a container from an image. It requires explicit flags for:

- Port mapping (`-p`)
- Volume mounting (`-v`)
- Environment variables (`-e`)
- Network configuration (`--network`)
- Restart policies (`--restart`)
- Resource limits (`--memory` , `--cpus`)
- Container name (`--name`)

This approach is **imperative**, meaning you specify step-by-step instructions.

Example:

```
docker run -d \  
  --name my-nginx \  
  -p 8080:80 \  
  -v ./html:/usr/share/nginx/html
```

```
-e NGINX_HOST=localhost \
--restart unless-stopped \
nginx:alpine
```

2.2 Docker Compose (Declarative Approach)

Docker Compose uses a YAML file (`docker-compose.yml`) to define services, networks, and volumes in a structured format.

Instead of multiple `docker run` commands, a single command is used:

```
docker compose up -d
```

Compose is **declarative**, meaning you define the desired state of the application.

Equivalent Compose file:

```
services:
  nginx:
    image: nginx:alpine
    container_name: my-nginx
    ports:
      - "8080:80"
    volumes:
      - ./html:/usr/share/nginx/html
    environment:
      NGINX_HOST: localhost
    restart: unless-stopped
```

3. Mapping: Docker Run vs Docker Compose

Docker Run Flag	Docker Compose Equivalent
<code>-p 8080:80</code>	<code>ports:</code>
<code>-v host:container</code>	<code>volumes:</code>
<code>-e KEY=value</code>	<code>environment:</code>
<code>--name</code>	<code>container_name:</code>
<code>--network</code>	<code>networks:</code>
<code>--restart</code>	<code>restart:</code>

Docker Run Flag	Docker Compose Equivalent
<code>--memory</code>	<code>deploy.resources.limits.memory</code>
<code>--cpus</code>	<code>deploy.resources.limits.cpus</code>
<code>-d</code>	<code>docker compose up -d</code>

4. Advantages of Docker Compose

1. Simplifies multi-container applications
2. Provides reproducibility
3. Version controllable configuration
4. Unified lifecycle management
5. Supports service scaling

Example:

```
docker compose up --scale web=3
```

B. TASK

Task 1: Single Container Comparison

Step 1: Run Nginx Using Docker Run

Execute:

```
docker run -d \  
  --name lab-nginx \  
  -p 8081:80 \  
  -v $(pwd)/html:/usr/share/nginx/html \  
  nginx:alpine
```

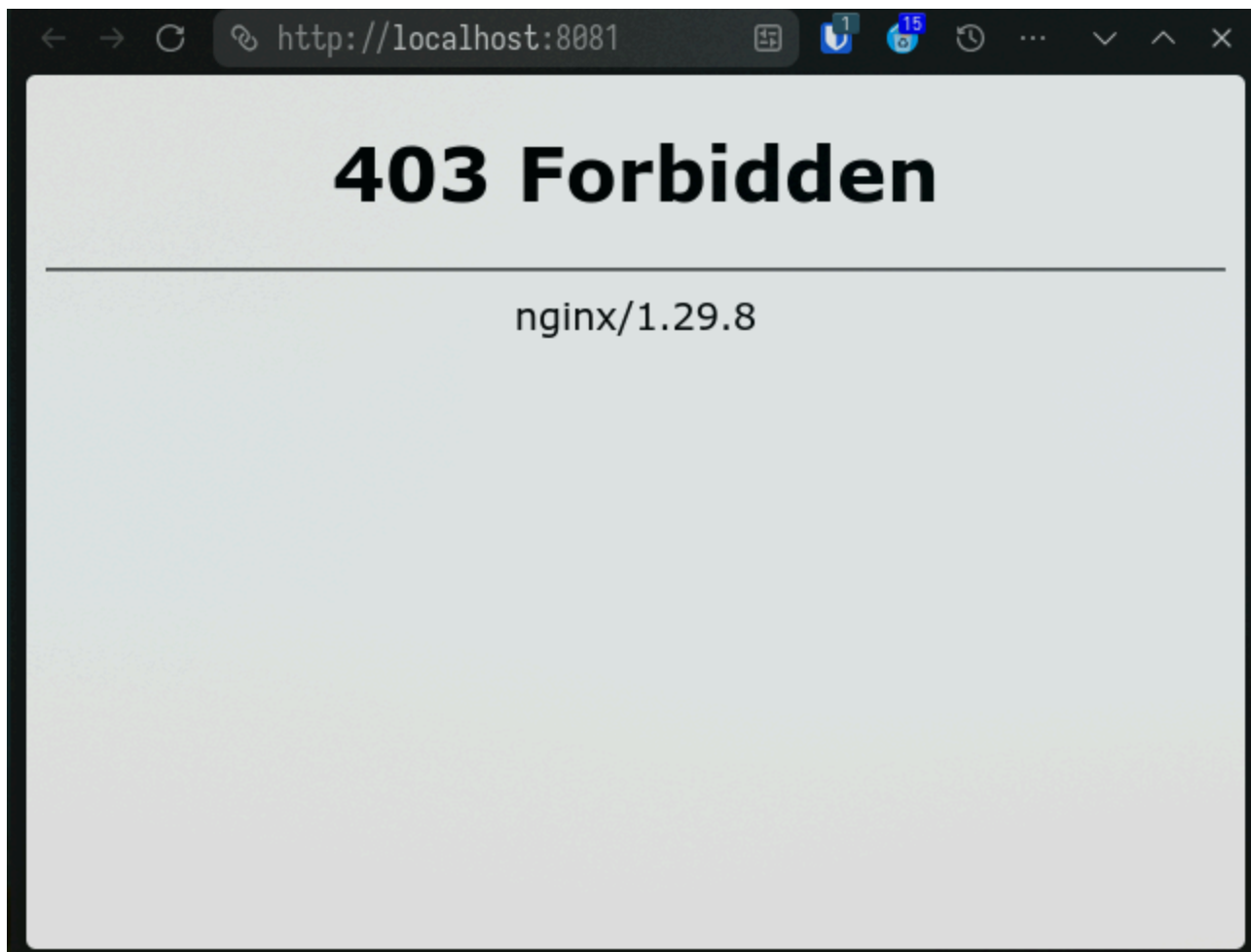
```
> docker run -d \
  --name lab-nginx \
  -p 8081:80 \
  -v $(pwd)/html:/usr/share/nginx/html \
  nginx:alpine
Unable to find image 'nginx:alpine' locally
alpine: Pulling from library/nginx
aee4e54b3865: Pull complete
6a0ac1617861: Pull complete
4a8b0b2a5b19: Pull complete
453da7dbc73e: Pull complete
583599bb7d38: Pull complete
781ff50d2644: Pull complete
612c0c1df4c5: Pull complete
82736a35d0e7: Pull complete
e8fc446e336c: Download complete
6192e1e6a438: Download complete
Digest: sha256:5616878291a2eed594aee8db4dade5878cf7edcb475e59193904b198d9b830de
Status: Downloaded newer image for nginx:alpine
497fc7b5603368fccab6f91c3d7531637e5c8f4c3ebbd0e4009cbb66c2899004
```

Verify:

```
docker ps
```

Access:

```
http://localhost:8081
```



Stop and remove container:

```
docker stop lab-nginx
docker rm lab-nginx
```

Step 2: Run Same Setup Using Docker Compose

Create `docker-compose.yml`:

```
services:
  nginx:
    image: nginx:alpine
    container_name: lab-nginx
    ports:
      - "8081:80"
    volumes:
      - ./html:/usr/share/nginx/html
```

Run:

```
docker compose up -d
```

Verify:

```
docker compose ps
```

Stop:

```
docker compose down
```

```
> docker compose up -d
[+] up 2/2
✓Network experiment6_default Created 0.1s
✓Container lab-nginx Started 0.4s
> docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS
lab-nginx           nginx:alpine        "/docker-entrypoint..." nginx      About a minute ago Up Abo
ut a minute        0.0.0.0:8081→80/tcp, [::]:8081→80/tcp
> docker compose down
[+] down 2/2
✓Container lab-nginx Removed 0.5s
✓Network experiment6_default Removed 0.1s
```

Task 2: Multi-Container Application

Objective:

Deploy WordPress with MySQL using:

1. Docker Run (manual way)
2. Docker Compose (structured way)

A. Using Docker Run

1. Create network:

```
docker network create wp-net
```

1. Run MySQL:

```
docker run -d \
  --name mysql \
  --network wp-net \
  -e MYSQL_DATABASE=wordpress \
  -e MYSQL_USER=wpuser \
  -e MYSQL_PASSWORD=secret \
  -e MYSQL_ROOT_PASSWORD=rootpass \
  mysql:latest
```

1. Run WordPress:

```
docker run -d \
  --name wordpress \
  --network wp-net \
  -p 8082:80 \
  -e WORDPRESS_DB_HOST=mysql \
  -e WORDPRESS_DB_NAME=wordpress \
  -e WORDPRESS_DB_USER=wpuser \
  -e WORDPRESS_DB_PASSWORD=secret \
  wordpress:latest
```

Test:

<http://localhost:8082>

B. Using Docker Compose

Create `docker-compose.yml`:

```
services:
  mysql:
    image: mysql:latest
    container_name: mysql
    environment:
      MYSQL_ROOT_PASSWORD: secret
      MYSQL_DATABASE: ${MYSQL_DATABASE}
      MYSQL_USER: ${MYSQL_USER}
      MYSQL_PASSWORD: ${MYSQL_PASSWORD}
    volumes:
      - mysql_data:/var/lib/mysql
    networks:
      - wordpress-network

wordpress:
```

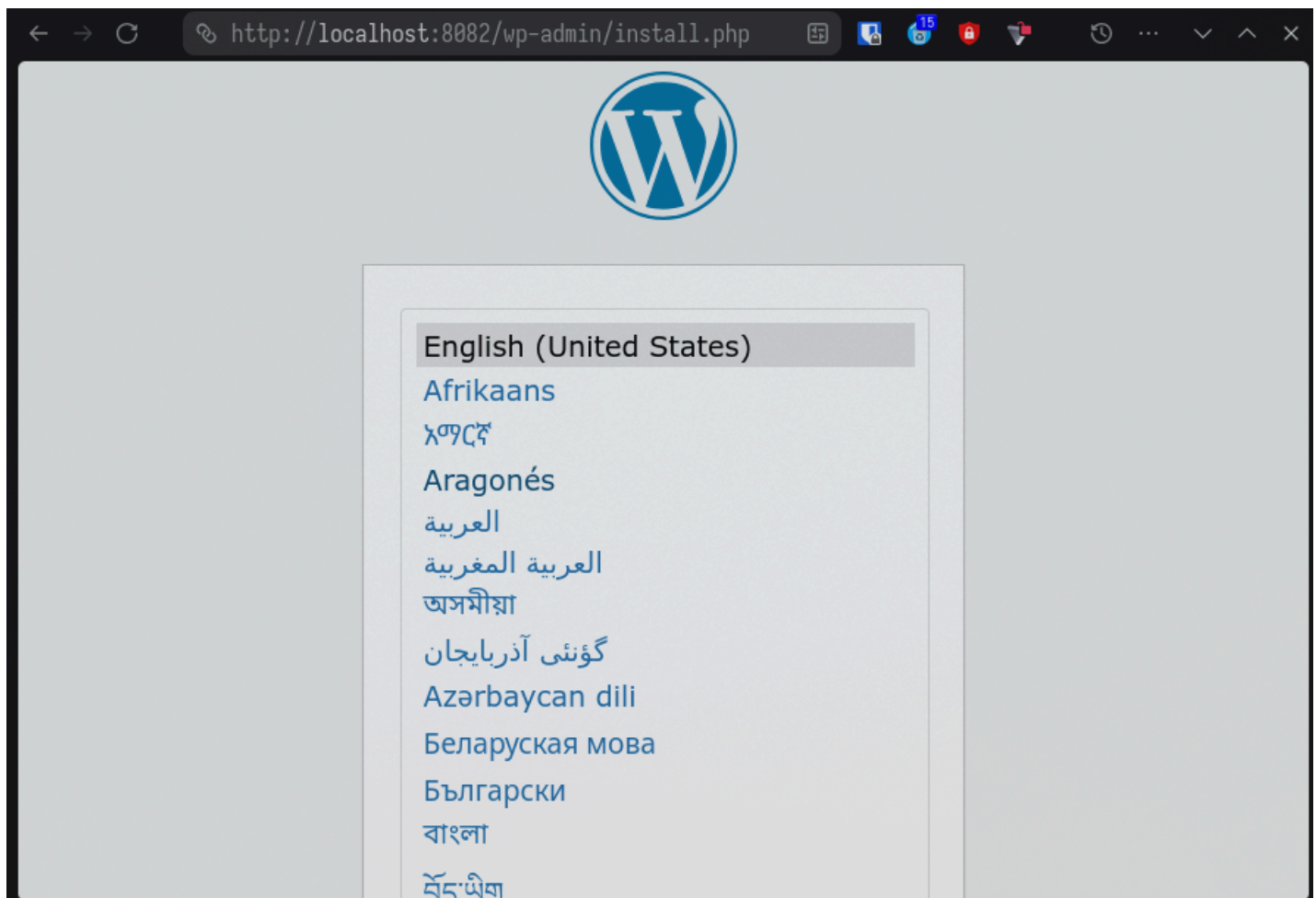
```
image: wordpress:latest
container_name: wordpress # Remove to scale
ports:
  - "8082:80"
environment:
  WORDPRESS_DB_HOST: mysql
  WORDPRESS_DB_NAME: ${MYSQL_DATABASE}
  WORDPRESS_DB_USER: ${MYSQL_USER}
  WORDPRESS_DB_PASSWORD: ${MYSQL_PASSWORD}
depends_on:
  - mysql
networks:
  - wordpress-network

volumes:
  mysql_data:

networks:
  wp-network:
```

Run:

```
docker compose up -d
```



Stop:

```
docker compose down -v
```

Containerization-Devops-Class

Lab Experiment 7: CI/CD using Jenkins, GitHub and Docker Hub

1. Aim

To design and implement a complete CI/CD pipeline using **Jenkins**, integrating source code from **GitHub**, and building & pushing Docker images to **Docker Hub**.

2. Objectives

- Understand CI/CD workflow using Jenkins (GUI-based tool)
- Create a structured GitHub repository with application + Jenkinsfile
- Build Docker images from source code
- Securely store Docker Hub credentials in Jenkins
- Automate build & push process using webhook triggers
- Use same host (Docker) as Jenkins agent

3. Theory

What is Jenkins?

Jenkins is a **web-based GUI automation server** used to:

- Build applications
- Test code
- Deploy software

It provides:

- Dashboard (browser-based UI)
- Plugin ecosystem (GitHub, Docker, etc.)
- Pipeline as Code using `Jenkinsfile`

What is CI/CD?

- **Continuous Integration (CI):** Code is automatically built and tested after each commit
- **Continuous Deployment (CD):** Built artifacts (Docker images) are automatically delivered/deployed

Workflow Overview

Developer → GitHub → Webhook → Jenkins → Build → Docker Hub

4. Prerequisites

- Docker & Docker Compose installed
- GitHub account
- Docker Hub account
- Basic Linux command knowledge

5. Part A: GitHub Repository Setup (Source Code + Build Definition)

5.1 Application Code

1. `app.py`

```
from flask import Flask
app = Flask(__name__)

@app.route("/")
def home():
    return "Hello from CI/CD Pipeline!"
    # return "Hello from CI/CD Pipeline!, my sapid is 500119624"

app.run(host="0.0.0.0", port=8000)
```

2. `requirements.txt`

```
blinker==1.9.0
click==8.3.1
Flask==3.1.3
itsdangerous==2.2.0
Jinja2==3.1.6
```

MarkupSafe==3.0.3
Werkzeug==3.1.7

5.2 Dockerfile (Build Process)

```
FROM python:3.13-slim

WORKDIR /app
COPY . .

RUN pip install -r requirements.txt

EXPOSE 80
CMD ["python", "app.py"]
```

5.3 Jenkinsfile (Pipeline Definition in GitHub)

```
pipeline {
    agent any

    environment {
        IMAGE_NAME = "rajsunny004/myapp"
    }

    stages {
        stage('Clone Source') {
            steps {
                git 'https://github.com/RajSunny2011/my-app.git'
            }
        }

        stage('Build Docker Image') {
            steps {
                sh 'docker build -t $IMAGE_NAME:$BUILD_NUMBER -t $IMAGE_NAME:latest .'
            }
        }

        stage('Login to Docker Hub') {
            steps {
                withCredentials([string(credentialsId: 'dockerhub-token', variable: 'DOCKER_TOKEN')]) {
                    sh 'echo $DOCKER_TOKEN | docker login -u rajsunny004 --password-s
                }
            }
        }
    }
}
```

```
    stage('Push to Docker Hub') {  
        steps {  
            sh 'docker push $IMAGE_NAME:$BUILD_NUMBER'  
            sh 'docker push $IMAGE_NAME:latest'  
        }  
    }  
}  
}
```

6. Part B: Jenkins Setup using Docker (Persistent Configuration)

6.1 Create Docker Compose File

```
services:  
  jenkins:  
    image: jenkins/jenkins:lts  
    container_name: jenkins  
    restart: always  
    ports:  
      - "8080:8080"  
      - "50000:50000"  
    volumes:  
      - jenkins_home:/var/jenkins_home  
      - /var/run/docker.sock:/var/run/docker.sock  
    user: root  
  
volumes:  
  jenkins_home:
```

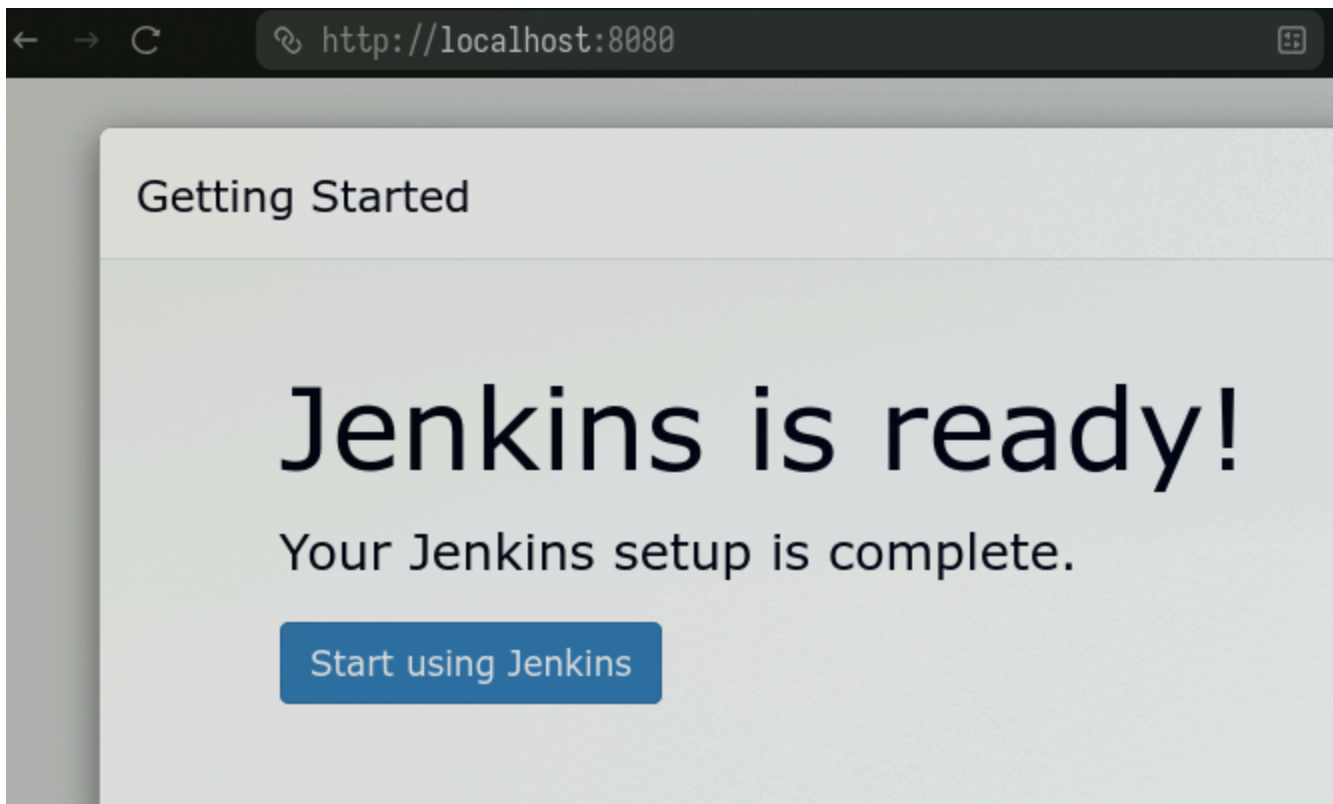
6.2 Start Jenkins

```
docker-compose up -d
```

```
> docker compose up -d
[+] up 17/17
✓ Image jenkins/jenkins:lts      Pulled
✓ Network experiment7_default     Created
✓ Volume experiment7_jenkins_home Created
✓ Container jenkins              Started
```

Access:

`http://localhost:8080`



6.3 Unlock Jenkins

```
docker exec -it jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

7. Part C: Jenkins Configuration

7.1 Add Docker Hub Credentials

Path:

Manage Jenkins → Credentials → Add Credentials

- Type: Secret Text
- ID: `dockerhub-token`
- Value: Docker Hub Access Token

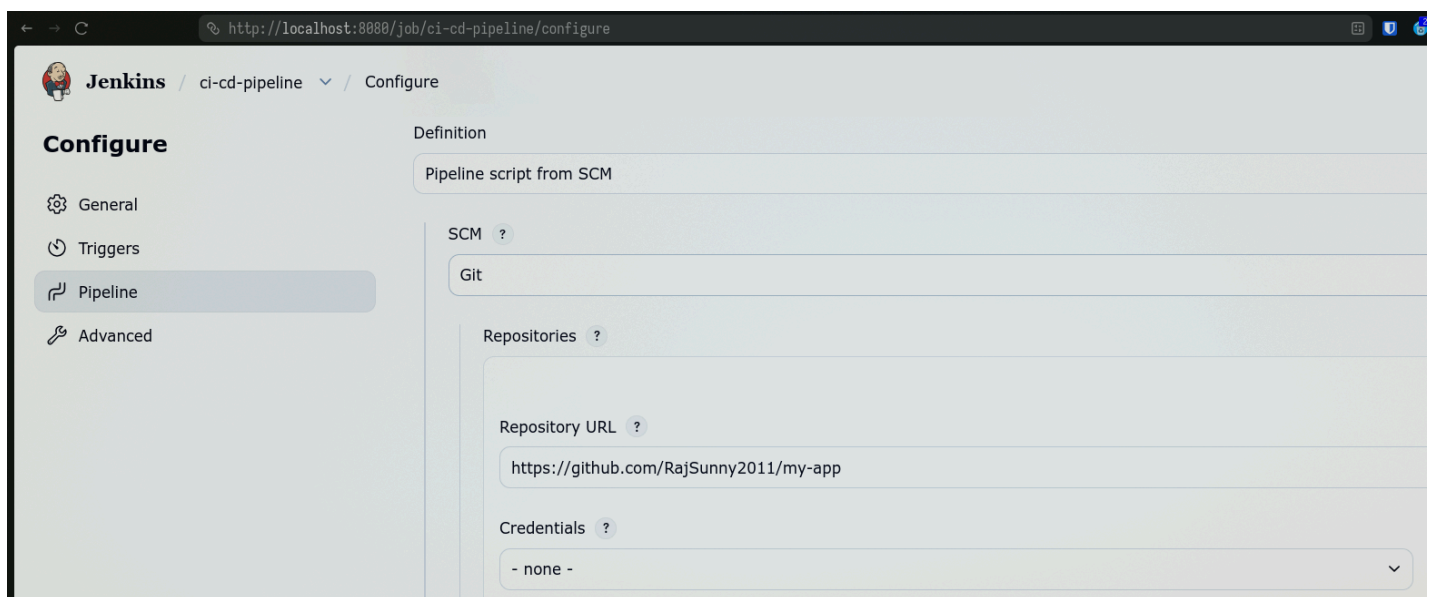
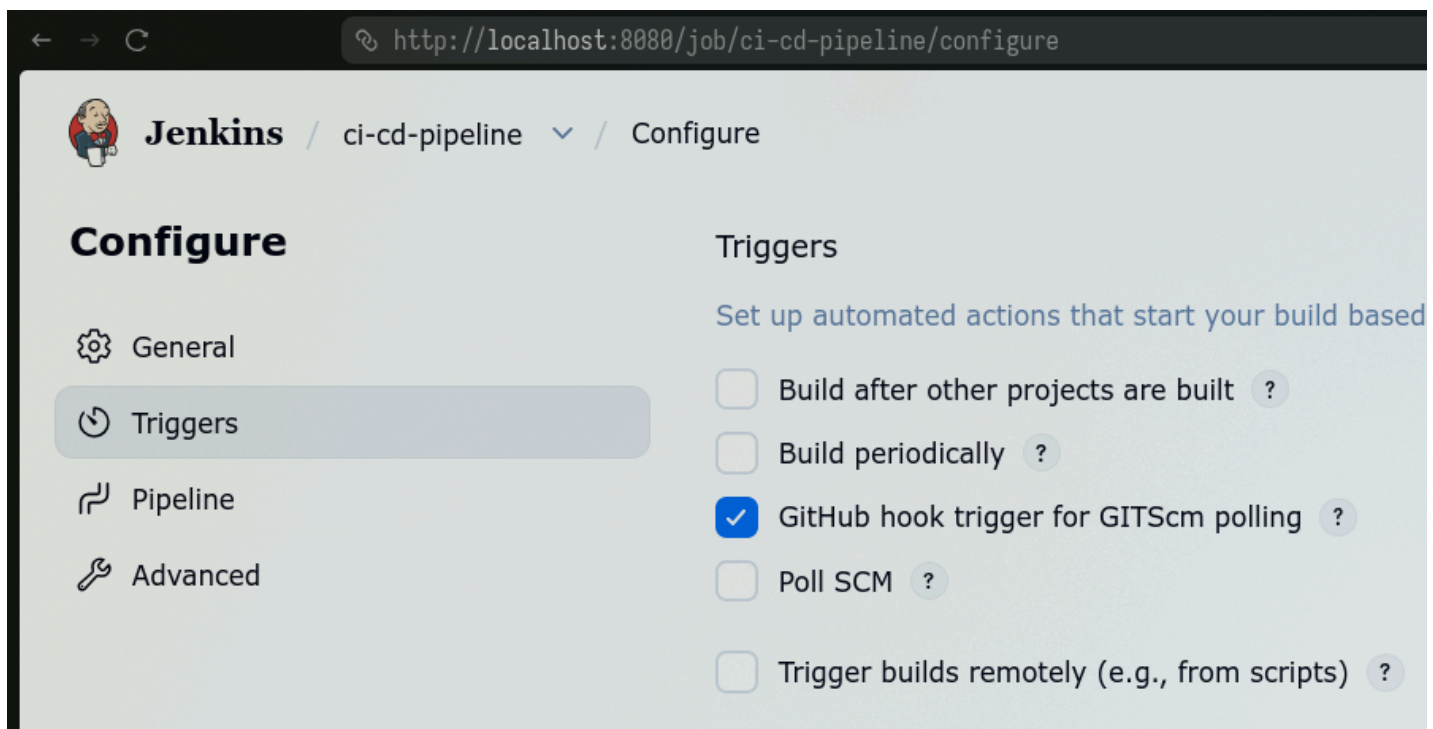
7.2 Create Pipeline Job

1. New Item → Pipeline
2. Name: `ci-cd-pipeline`

Configure:

Pipeline script from SCM

- SCM: Git
- Repo URL: your GitHub repo
- Script Path: `Jenkinsfile`



8. Part D: GitHub Webhook Integration

8.1 Configure Webhook

In GitHub:

Settings → Webhooks → Add Webhook

Payload URL:

http://<your-server-ip>:8080/github-webhook/

Events:

Push events

Webhooks / Add webhook

We'll send a POST request to the URL below with details of any subscribed events. You can also specify which data format you'd like to receive (JSON, x-www-form-urlencoded, etc). More information can be found in [our developer documentation](#).

Payload URL *
https://better-feet-hammer.localt.it/github-webhook/

Content type *
application/json

Secret

SSL verification
By default, we verify SSL certificates when delivering payloads.

☒ **Enable SSL verification** ☐ Disable (not recommended)

Which events would you like to trigger this webhook?

☒ Just the push event.
☐ Send me **everything**.
☐ Let me select individual events.

☒ **Active**
We will deliver event details when this hook is triggered.

Result:



Status

</> Changes

📄 Console Output

✍ Edit Build Information

🗑 Delete build '#3'

🕒 Timings

📦 Git Build Data

🔗 Pipeline Overview

🔄 Restart from Stage

↺ Replay

☰ Pipeline Steps

📁 Workspaces



#3 (Mar 28, 2026, 9:35:33 AM)



Started by user [Sativik Raj](#)



This run spent:

- 11 ms waiting;
- 1 min 9 sec build duration;
- 1 min 9 sec total from scheduled to completion.



Revision: cd8a16e882935ce6bc75c0c2e89b9806add721d
Repository: <https://github.com/RajSunny2011/my-app>

- refs/remotes/origin/master



No changes.



Status

</> Changes

📄 Console Output

✍ Edit Build Information

📄 Polling Log

🕒 Timings

📦 Git Build Data

🔗 Pipeline Overview

📄 Thread Dump



#5 (Mar 28, 2026, 9:45:47 AM)



Started by GitHub push by [RajSunny2011](#)



This run spent 7.4 sec waiting in the queue.



Revision: 4cd6ff9562b1f920eb04407c11ac3585d6d55f1b
Repository: <https://github.com/RajSunny2011/my-app>

- refs/remotes/origin/master



No changes.

Containerization-Devops-Class

Experiment 9 : Ansible

Theory

Problem Statement: Managing infrastructure manually across multiple servers leads to configuration drift, inconsistent environments, and time-consuming repetitive tasks. Scaling from one server to hundreds becomes nearly impossible with manual SSH-based administration.

What is Ansible?

- Ansible is an open-source **automation tool** for **configuration management**, **application deployment**, and **orchestration**.
- It follows an **agentless** architecture, using SSH for Linux and WinRM for Windows.
- Uses YAML-based **playbooks** to define automation tasks.

How Ansible Solves the Problem:

- **Agentless Architecture:** No software installation required on managed nodes
- **Idempotency:** Running playbooks multiple times yields same result
- **Declarative Syntax:** Describe desired state, not the steps to achieve it
- **Push-based:** Initiates changes from control node immediately

Key Concepts

Component	Description
Control Node	Machine with Ansible installed
Managed Nodes	Target servers (no Ansible agent needed)
Inventory	Defines the list of managed nodes (EC2 instances, servers, etc.). <code>inventory.ini</code>
Playbooks	YAML files containing a sequence of automation steps.

Component	Description
Tasks	Individual actions in playbooks (e.g., installing a package).
Modules	Built-in functionality to perform tasks (e.g., <code>yum</code> , <code>apt</code> , <code>service</code>).
Roles	Pre-defined reusable automation scripts.

How does Ansible work?

Ansible uses the concepts of control and managed nodes. It connects from the **control node**, any machine with Ansible installed, to the **managed nodes** sending commands and instructions to them. The units of code that Ansible executes on the managed nodes are called **modules**. Each module is invoked by a **task**, and an ordered list of tasks together forms a **playbook**. Users write playbooks with tasks and modules to define the desired state of the system. The managed machines are represented in a simplistic **inventory** file that groups all the nodes into different categories. Ansible leverages a very simple language, [YAML](#), to define playbooks in a human-readable data format that is really easy to understand from day one.

Benefits of using Ansible

- A free and open-source community project with a huge audience.
- Battle-tested over many years as the preferred tool of IT wizards.
- Easy to start and use from day one, without the need for any special coding skills.
- Simple deployment workflow without any extra agents.
- Includes some sophisticated features around modularity and reusability that come in handy as users become more proficient.
- Extensive and comprehensive official documentation that is complemented by a plethora of online material produced by its community.

Part A

Ansible Installation Instructions

1. Install via `pip` (Python Package Manager)

```
# Install Ansible globally  
pip install ansible
```

```
# Verify installation
ansible --version
```

Best for: Latest versions, macOS/Linux, or Python environments.

2. Install via `apt` (Debian/Ubuntu)

```
# Update packages
sudo apt update -y

# Install Ansible
sudo apt install ansible -y

# Verify
ansible --version
```

Create Docker image and test ssh login

First create ssh key-pair and then create custom ubuntu-server image with open-ssh configured.

Testing SSH Key Pair Login with Docker and WSL

1. Create SSH Key Pair

First, generate an SSH key pair:

```
# Generate RSA key pair (accept defaults when prompted)
ssh-keygen -t rsa -b 4096

# This creates:
# Private key: ~/.ssh/id_rsa
# Public key: ~/.ssh/id_rsa.pub
# copy keys to current directory to be added to docker images
cp ~/.ssh/id_rsa.pub .
cp ~/.ssh/id_rsa .
```

```

> ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (/home/sunny/.ssh/id_rsa):
Enter passphrase for "/home/sunny/.ssh/id_rsa" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/sunny/.ssh/id_rsa
Your public key has been saved in /home/sunny/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:6w7fub+3UD3Kpi6elnHSBf37qkjj4fj/5WvZ6ubgcK0 sunny@cachyos-A15
The key's randomart image is:
+---[RSA 4096]-----+
|           .           |
|          . .          |
|         . .           |
|        . o            |
|       S . . ..o       |
|      + o.o...         |
|     . . X ++. =       |
|    + X.Oo+o=o         |
|   .B+@*E00=o         |
+-----[SHA256]-----+

```

2. Create Dockerfile for Ubuntu SSH Server

Create a Dockerfile with the following content:

```

FROM ubuntu

RUN apt update -y
RUN apt install -y python3 python3-pip openssh-server
RUN mkdir -p /var/run/sshd

# Configure SSH
RUN mkdir -p /run/sshd && \
    echo 'root:password' | chpasswd && \
    sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/' /etc/ssh/sshd_
    sed -i 's/#PasswordAuthentication yes/PasswordAuthentication no/' /etc/ssh/sshd_c
    sed -i 's/#PubkeyAuthentication yes/PubkeyAuthentication yes/' /etc/ssh/sshd_conf

# Create .ssh directory and set proper permissions
RUN mkdir -p /root/.ssh && \
    chmod 700 /root/.ssh

# Copy SSH keys (note: this is not secure for production!)
COPY id_rsa /root/.ssh/id_rsa

```

```

COPY id_rsa.pub /root/.ssh/authorized_keys

# Set proper permissions for keys
RUN chmod 600 /root/.ssh/id_rsa && \
    chmod 644 /root/.ssh/authorized_keys

# Fix for SSH login
RUN sed -i 's@session\s*required\s*pam_loginuid.so@session optional pam_loginuid.so@g

# Expose SSH port
EXPOSE 22

# Start SSH service when container starts
CMD ["/usr/sbin/sshd", "-D"]

```

3. Build the Docker Image

```

# Copy your public key to the current directory
cp ~/.ssh/id_rsa.pub .

# Build the Docker image
docker build -t ubuntu-server .

# Remove the public key from build directory (optional)
rm id_rsa.pub

```

4. Run the Docker Container

```

# Run the container with port mapping
docker run -d -p --rm 2222:22 -p 8221:8221 --name ssh-test-server ubuntu-server

```

5. Find the Container IP Address

```

# Get the container's IP address
docker inspect -f '{{.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' ssh-test-server

```

```

> docker run -d --rm -p 2222:22 -p 8221:8221 --name ssh-test-server ubuntu-server
5eed3d4daf28fe7817fcd3b7fad1d1cea8e917c642946d4e2887f56dda1686d6
> docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' ssh-test-server
172.17.0.2

```

6. Test SSH Connections

Test password authentication:

```
ssh root@localhost -p 2222
# OR
ssh root@172.17.0.2
```

Test key-based authentication:

```
ssh -i ~/.ssh/id_rsa root@localhost -p 2222
```

Clean Up

When done testing:

```
docker stop ssh-test
docker rm ssh-test
```

Ansible with Docker Exercise

Using docker image `ubuntu-server` created in previous part run 4 test servers named server1 to server4

Step 1: Start multiple containers to act as server (to be configured by ansible)

```
for i in {1..4}; do
  echo -e "\n Creating server${i}\n"
  docker run -d --rm -p 220${i}:22 --name server${i} ubuntu-server
  echo -e "IP of server${i} is $(docker inspect -f ' ' server${i})"
done
```

```
>>
) for i in {1..4}; do
  echo -e "\n Creating server${i}\n"
  docker run -d --rm -p 220${i}:22 --name server${i} ubuntu-server
  echo -e "IP of server${i} is $(docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' server${i})"
done

Creating server1
5b0605f9b6e6060611537caecd03aaf08199401ff7d411a7f43b4de3025cc31b
IP of server1 is 172.17.0.3

Creating server2
9785486150da6293ee5aea910f7782ff407fa880aaf3a7c0d7b56ef8e8828768
IP of server2 is 172.17.0.4

Creating server3
bc04bf8dd7c0017e1e7ab011d7e562bf9018745c69ed23990dfaeb072a13c856
IP of server3 is 172.17.0.5

Creating server4
ac3e116dfe6cc35edeacc30c548fa1a03c575fb2699d34fe0a9d741c4522feaa
IP of server4 is 172.17.0.6
```

Step 2: Create Ansible Inventory

below script will create an update inventory.ini with updated docker container IPs, review this file and you can create your own if needed.

```
# Get container IPs
echo "[servers]" > inventory.ini
for i in {1..4}; do
  docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' server${i} >> inventory.ini
done

# Add inventory variables
cat << EOF >> inventory.ini

[servers:vars]
ansible_user=root
; ansible_ssh_private_key_file=./id_rsa
ansible_ssh_private_key_file=~/.ssh/id_rsa
ansible_python_interpreter=/usr/bin/python3
EOF
```

Step 3: review content of inventory.ini

```
cat inventory.ini
```

```

) echo "[servers]" > inventory.ini
for i in {1..4}; do
    docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' server${i} >> inventory.ini
done

) cat << EOF >> inventory.ini

[servers:vars]
ansible_user=root
; ansible_ssh_private_key_file=./id_rsa
ansible_ssh_private_key_file=~/.ssh/id_rsa
ansible_python_interpreter=/usr/bin/python3
EOF

) cat inventory.ini

[servers]
172.17.0.3
172.17.0.4
172.17.0.5
172.17.0.6

[servers:vars]
ansible_user=root
; ansible_ssh_private_key_file=./id_rsa
ansible_ssh_private_key_file=~/.ssh/id_rsa
ansible_python_interpreter=/usr/bin/python3

```

Step 4: Test Connectivity

```

# Manual SSH test
ssh -i $(pwd)/id_rsa root@172.17.0.3
## above on will work on linux and may cause problems on NTFS file systems, then try
ssh -i ~/.ssh/id_rsa root@172.17.0.3

# Ansible ping test
ansible all -i inventory.ini -m ping
OR
ansible all -i inventory.ini -m ping -vvv

```

```
> ansible all -i inventory.ini -m ping
172.17.0.3 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
172.17.0.4 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
172.17.0.5 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
172.17.0.6 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
```

Step 5: Create Playbook (update.yml)

The yaml file should start with three dash only `---`

```
--- # it should start with three dash only
- name: Update and configure servers
  hosts: all
  become: yes

  tasks:
    - name: Update apt packages
      apt:
        update_cache: yes
        upgrade: dist

    - name: Install required packages
      apt:
        name: ["vim", "htop", "wget"]
        state: present

    - name: Create test file
      copy:
        dest: /root/ansible_test.txt
        content: "Configured by Ansible on "
```

Step 6: Run Playbook

```
ansible-playbook -i inventory.ini playbook1.yml
```

```
> ansible-playbook -i inventory.ini playbook1.yml

PLAY [Update and configure servers] *****

TASK [Gathering Facts] *****
ok: [172.17.0.6]
ok: [172.17.0.3]
ok: [172.17.0.4]
ok: [172.17.0.5]

TASK [Update apt packages] *****
ok: [172.17.0.6]
ok: [172.17.0.4]
ok: [172.17.0.5]
ok: [172.17.0.3]

TASK [Install required packages] *****
changed: [172.17.0.6]
changed: [172.17.0.3]
changed: [172.17.0.4]
changed: [172.17.0.5]

TASK [Create test file] *****
changed: [172.17.0.6]
changed: [172.17.0.5]
changed: [172.17.0.3]
changed: [172.17.0.4]

PLAY RECAP *****
172.17.0.3      : ok=4    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
172.17.0.4      : ok=4    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
172.17.0.5      : ok=4    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
172.17.0.6      : ok=4    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Step 7: Verify Changes

```
# Using Ansible
ansible all -i inventory.ini -m command -a "cat /root/ansible_test.txt"

# Manually via Docker
for i in {1..4}; do
  docker exec server${i} cat /root/ansible_test.txt
done
```

```
> ansible all -i inventory.ini -m command -a "cat /root/ansible_test.txt"
172.17.0.4 | CHANGED | rc=0 >>
Configured by Ansible on 172.17.0.4
172.17.0.5 | CHANGED | rc=0 >>
Configured by Ansible on 172.17.0.5
172.17.0.6 | CHANGED | rc=0 >>
Configured by Ansible on 172.17.0.6
172.17.0.3 | CHANGED | rc=0 >>
Configured by Ansible on 172.17.0.3
```

Step 8: Cleanup

```
# Stop and remove containers
```

```
for i in {1..4}; do docker rm -f server${i}; done
```

Containerization-Devops-Class

Experiment 10: SonarQube — Static Code Analysis

Step 1: Start the SonarQube Server

We will use Docker Compose to start both the SonarQube server and its PostgreSQL database together.

Create a file called `docker-compose.yml` :

```
# docker-compose.yml
# This file starts two containers:
# 1. sonar-db      → PostgreSQL database (stores SonarQube data)
# 2. sonarqube     → The SonarQube server (web UI + analysis engine)
services:

# — Database —————
sonar-db:
  image: postgres:13
  container_name: sonar-db
  environment:
    POSTGRES_USER: sonar          # DB username
    POSTGRES_PASSWORD: sonar      # DB password
    POSTGRES_DB: sonarqube        # DB name
    POSTGRES_HOST_AUTH_METHOD: trust # allow connections without password (for sim
  volumes:
    - sonar-db-data:/var/lib/postgresql/data # persist data across restarts
  networks:
    - sonarqube-lab

# — SonarQube Server —————
sonarqube:
  image: sonarqube:lts-community
  container_name: sonarqube
  ports:
    - "9000:9000" # access dashboard at http://localhost:9000
  environment:
    SONAR_JDBC_URL: jdbc:postgresql://sonar-db:5432/sonarqube
    SONAR_JDBC_USERNAME: sonar
    SONAR_JDBC_PASSWORD: sonar
```

```
volumes:
  - sonar-data:/opt/sonarqube/data
  - sonar-extensions:/opt/sonarqube/extensions
depends_on:
  - sonar-db          # wait for DB to start first
networks:
  - sonarqube-lab
```

Named volumes (Docker manages storage location)

```
volumes:
  sonar-db-data:
  sonar-data:
  sonar-extensions:
```

Isolated network so containers can talk to each other by name

```
networks:
  sonarqube-lab:
    driver: bridge
```

Start both containers:

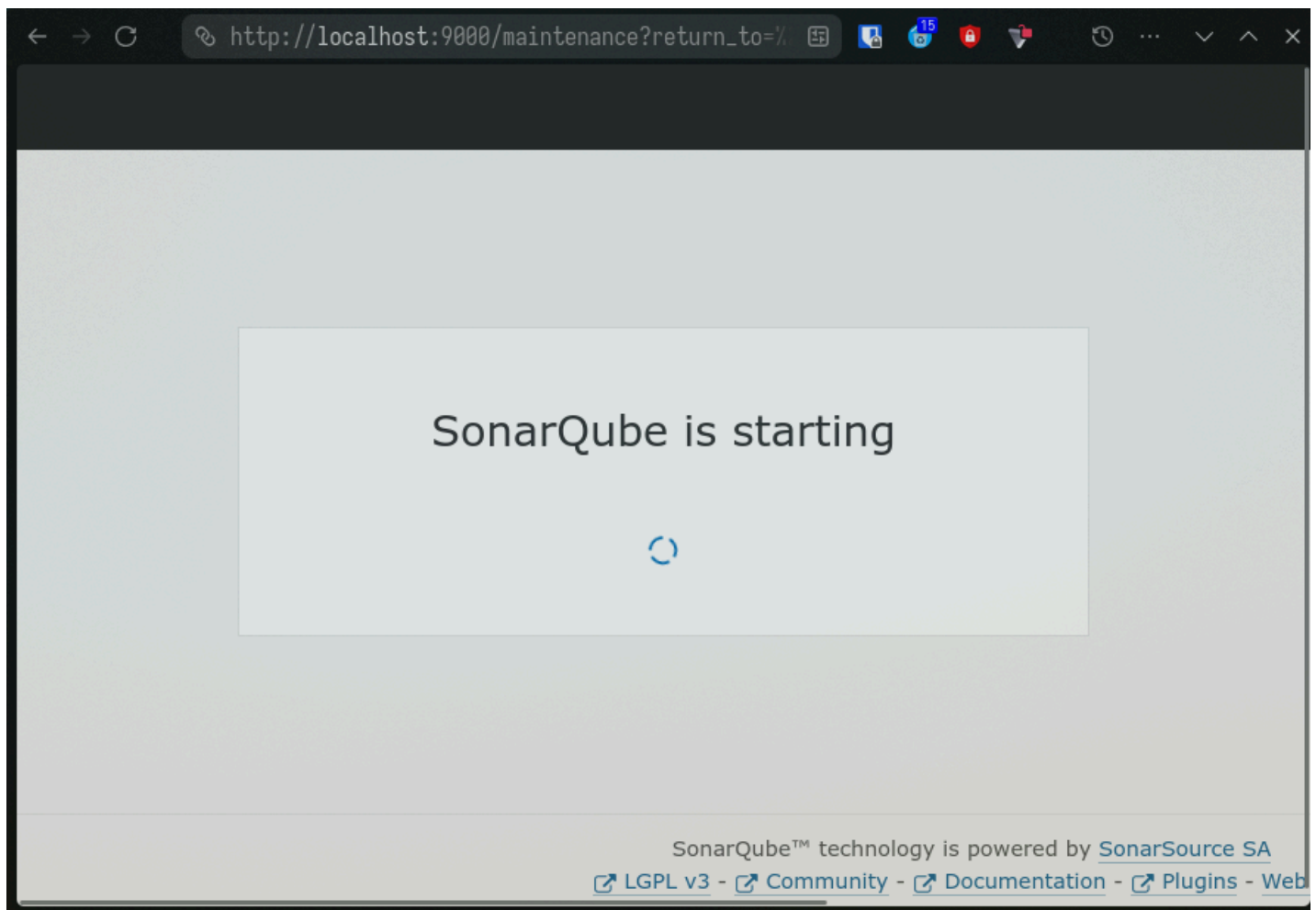
```
docker-compose up -d
```

Watch the logs until you see "SonarQube is up"

```
docker-compose logs -f sonarqube
```

```
> docker-compose up -d
[+] up 6/6
✓Network experiment10_sonarqube-lab    Created    0.1s
✓Volume experiment10_sonar-data        Created    0.0s
✓Volume experiment10_sonar-extensions Created    0.0s
✓Volume experiment10_sonar-db-data     Created    0.0s
✓Container sonar-db                    Started    0.7s
✓Container sonarqube                   Started    0.6s
```

Once started, open `http://localhost:9000` in your browser. Default login: **admin / admin** (you will be asked to change this on first login).



Step 2: Create a Sample Java App with Code Issues

```
mkdir -p sample-java-app/src/main/java/com/example
cd sample-java-app
```

Create `src/main/java/com/example/Calculator.java` :

```
package com.example;

public class Calculator {

    // — BUG: Division by zero is not handled —————
    // If someone calls divide(5, 0), this will crash at runtime.
    public int divide(int a, int b) {
        return a / b;
    }

    // — CODE SMELL: Unused variable —————
    // The variable 'unused' is declared but never used.
    // SonarQube flags this as unnecessary clutter.
```

```

public int add(int a, int b) {
    int result = a + b;
    int unused = 100;    // ← code smell: delete this line
    return result;
}

// — VULNERABILITY: SQL Injection risk —————
// Building a query by concatenating user input is dangerous.
// An attacker could pass: "1 OR 1=1" and get all users.
public String getUser(String userId) {
    String query = "SELECT * FROM users WHERE id = " + userId;
    return query;
}

// — CODE SMELL: Duplicated code —————
// The two methods below do exactly the same thing.
// This is copy-paste code and should be a single method.
public int multiply(int a, int b) {
    int result = 0;
    for (int i = 0; i < b; i++) {
        result = result + a;
    }
    return result;
}

public int multiplyAlt(int a, int b) {
    int result = 0;
    for (int i = 0; i < b; i++) {
        result = result + a;    // ← exact duplicate of multiply()
    }
    return result;
}

// — BUG: Null pointer risk —————
// If 'name' is null, calling .toUpperCase() will throw
// a NullPointerException at runtime.
public String getName(String name) {
    return name.toUpperCase();
}

// — CODE SMELL: Empty catch block —————
// The exception is caught but silently ignored.
// This hides errors and makes debugging very hard.
public void riskyOperation() {
    try {
        int x = 10 / 0;
    } catch (Exception e) {
        // ← never leave catch blocks empty
    }
}

```

```
}  
}
```

Create `pom.xml` (Maven build file):

```
<?xml version="1.0" encoding="UTF-8"?>  
<project xmlns="http://maven.apache.org/POM/4.0.0"  
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0  
        http://maven.apache.org/xsd/maven-4.0.0.xsd">  
  
    <modelVersion>4.0.0</modelVersion>  
  
    <!-- Project identity -->  
    <groupId>com.example</groupId>  
    <artifactId>sample-app</artifactId>  
    <version>1.0-SNAPSHOT</version>  
  
    <properties>  
        <maven.compiler.source>11</maven.compiler.source>  
        <maven.compiler.target>11</maven.compiler.target>  
  
        <!-- SonarQube connection settings -->  
        <sonar.projectKey>sample-java-app</sonar.projectKey>  
        <sonar.host.url>http://localhost:9000</sonar.host.url>  
        <!-- Replace with your actual token (generated in Step 3) -->  
        <sonar.login>YOUR_TOKEN_HERE</sonar.login>  
    </properties>  
  
    <dependencies>  
        <!-- JUnit for unit tests -->  
        <dependency>  
            <groupId>junit</groupId>  
            <artifactId>junit</artifactId>  
            <version>4.13.2</version>  
            <scope>test</scope>  
        </dependency>  
    </dependencies>  
  
    <build>  
        <plugins>  
            <!-- This plugin lets us run: mvn sonar:sonar -->  
            <plugin>  
                <groupId>org.sonarsource.scanner.maven</groupId>  
                <artifactId>sonar-maven-plugin</artifactId>  
                <version>3.9.1.2184</version>  
            </plugin>  
        </plugins>
```

```
</build>
```

```
</project>
```

Step 3: Generate a Token (Manual UI Step)

The Scanner needs a token to authenticate with the server. You generate this in the web UI.

1. Open `http://localhost:9000`
2. Log in as admin
3. Click your user icon (top right) → "My Account"
4. Click the "Security" tab
5. Under "Generate Tokens", type a name: `scanner-token`
6. Click "Generate"
7. Copy the token immediately – it is shown only once!

Step 4: Run the Scanner

There are two ways to run the scanner. Pick the one that matches your setup.

Using the Maven Plugin (recommended for Java)

```
# Make sure you are inside the sample-java-app folder
cd sample-java-app

# Run the SonarQube analysis via Maven
# Replace YOUR_TOKEN with the token copied in Step 3
mvn sonar:sonar -Dsonar.login=YOUR_TOKEN
```

```
j) mvn sonar:sonar -Dsonar.login=sqa_c7088ab6ef6feb8c868e980522b3e5be4c570a45
[INFO] Scanning for projects...
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-metadata.xml
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/mojo/maven-metadata.xml
Downloading from central: https://repo.maven.apache.org/maven2/org/sonarsource/scanner/maven/maven-metadata.xml
Downloaded from central: https://repo.maven.apache.org/maven2/org/sonarsource/scanner/maven/maven-metadata.xml (240 B at 501 B/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-metadata.xml (14 kB at 30 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/mojo/maven-metadata.xml (21 kB at 38 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/sonarsource/scanner/maven/sonar-maven-plugin/3.9.1.2184/sonar-maven-plugin-3.9.1.2184.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/sonarsource/scanner/maven/sonar-maven-plugin/3.9.1.2184/sonar-maven-plugin-3.9.1.2184.pom (12 kB at 173 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/sonarsource/parent/parent/59.0.29/parent-59.0.29.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/sonarsource/parent/parent/59.0.29/parent-59.0.29.pom (32 kB at 495 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/sonarsource/scanner/maven/sonar-maven-plugin/3.9.1.2184/sonar-maven-plugin-3.9.1.2184.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/sonarsource/scanner/maven/sonar-maven-plugin/3.9.1.2184/sonar-maven-plugin-3.9.1.2184.jar (50 kB at 776 kB/s)
[INFO]
[INFO] -----< com.example:sample-app >-----
[INFO] Building sample-app 1.0-SNAPSHOT
[INFO]   from pom.xml
[INFO] -----[ jar ]-----
```

Step 5: View Results

`http://localhost:9000/dashboard?id=sample-java-app`

MEASURES

New Code

Overall Code

1

 Bugs

Reliability

D

0

 Vulnerabilities

Security

A

1

 Security Hotspots 

0.0%

Reviewed

Security Review

E

57min

Debt

6

 Code Smells

Maintainability

A

0.0%

Coverage on 21 Lines to cover

-

Unit Tests

0.0%

Duplications on 72 Lines

0

Duplicated Blocks

Containerization-Devops-Class

Experiment 11: Orchestration using Docker Compose & Docker Swarm

Prerequisites

- Docker installed (with Swarm mode enabled)
- The `docker-compose.yml` file from Experiment 6 (WordPress + MySQL)

```
services:
  db:
    image: mysql:latest
    container_name: wordpress_db
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: wppass
    volumes:
      - db_data:/var/lib/mysql

  wordpress:
    image: wordpress:latest
    container_name: wordpress_app
    depends_on:
      - db
    ports:
      - "8080:80"
    restart: always
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wpuser
      WORDPRESS_DB_PASSWORD: wppass
      WORDPRESS_DB_NAME: wordpress
    volumes:
      - wp_data:/var/www/html
```

```
volumes:
```

```
db_data:
wp_data:
```

Task 1: Check Current State (No Swarm)

```
docker compose down -v

docker ps
```

Task 2: Initialize Docker Swarm

```
docker swarm init
```

Verify Swarm is active:

```
docker node ls
```

```
> docker swarm init
Swarm initialized: current node (vsw7yty8eo46c716evkazblau) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-31l9ynq3iww4ga8hsu1438416nlrin84tx05905sb619tekyuk-ck2qsyr5qrm1p5ps9axb76drw 10.230.65.168:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.
> docker node ls
ID                                HOSTNAME        STATUS    AVAILABILITY    MANAGER STATUS    ENGINE VERSION
vsw7yty8eo46c716evkazblau *      cachyos-A15     Ready     Active           Leader             29.4.0
```

Task 3: Deploy as a Stack (Not Just Compose)

```
docker stack deploy -c docker-compose.yml wpstack
```

```
> docker stack deploy -c docker-compose.yml wpstack
Ignoring unsupported options: restart

Ignoring deprecated options:

container_name: Setting the container name is not supported.

Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network wpstack_default
Creating service wpstack_db
Creating service wpstack_wordpress
```

Task 4: Verify the Deployment

List all services in the stack:

```
docker service ls
```

```
> docker service ls
```

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
ogt1mk70gaxl	wpstack_db	replicated	1/1	mysql:latest	
27kx76gz69uw	wpstack_wordpress	replicated	1/1	wordpress:latest	*:8080→80/tcp

```
docker service ps wpstack_wordpress
```

Task 5: Access WordPress

Open your browser:

```
http://localhost:8080
```

Task 6: Scale the Application (Swarm's Superpower)

Scale WordPress from 1 to 3 replicas:

```
docker service scale wpstack_wordpress=3
```

```
> docker service scale wpstack_wordpress=3
wpstack_wordpress scaled to 3
overall progress: 3 out of 3 tasks
1/3: running [=====>]
2/3: running [=====>]
3/3: running [=====>]
verify: Service wpstack_wordpress converged
```

Verify scaling:

```
docker service ls
```

```
> docker service ls
ID                NAME                MODE                REPLICAS            IMAGE                PORTS
ogt1mk70gaxl     wpstack_db          replicated           1/1                 mysql:latest
27kx76gz69uw     wpstack_wordpress   replicated           3/3                 wordpress:latest     *:8080→80/tcp
```

Notice the REPLICAS column shows `3/3` for wordpress.

Quick Reference Card

```
# Initialize Swarm
docker swarm init

# Deploy stack
docker stack deploy -c docker-compose.yml <stack-name>

# List services
docker service ls

# Scale service
docker service scale <stack-name_service-name>=<replicas>

# See service tasks
docker service ps <service-name>

# Remove stack
docker stack rm <stack-name>

# Leave Swarm (if needed)
docker swarm leave --force
```